

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	M.Mary Sharlin	Reg. No.:	192511330
Section / Batch:	BE CSE	Date:	22/07/2026

Code of Conduct

I M.Mary Sharlin (192511330) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Mary Sharlin

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 – 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key) {
4     int low = 0, high = n - 1;
5
6     while (low <= high) {
7         int mid = (low + high) / 2;
8
9         if (arr[mid] == key)
10             return mid;
11         else if (arr[mid] < key)
12             low = mid + 1;
13         else
14             high = mid - 1;
15     }
16     return -1;
17 }
18
19 int main() {
20     int arr[] = {10, 20, 30, 40, 50};
21     int key = 30, n = 5;
22 }
```

Test Input

stdin input (optional)

Output

Index = 2

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5], top = 0;
4
5 int pop() {
6     if (top == 0) {
7         printf("Underflow");
8         return -1;
9     }
10    return stack[--top];
11 }
12
13 int main() {
14     stack[0] = 10;
15     stack[1] = 20;
16     top = 2;
17
18     printf("Deleted: %d", pop());
19     return 0;
20 }
```

Test Input

stdin input (optional)

Output

Deleted: 20

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top;
5
6 void init() {
7     top = 0;
8 }
9
10 int main() {
11     init();
12     printf("Top = %d", top);
13     return 0;
14 }
```



Run



Save



Submit Fix

Test Input

stdin input (optional)

Output

Top = 0

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4 #include <string.h>
5
6 #define MAX 100
7
8 char stack[MAX];
9 int top = -1;
10
11 // Push element onto stack
12 void push(char item) {
13     if (top >= MAX - 1) {
14         printf("Stack Overflow\n");
15         return;
16     }
17     stack[++top] = item;
18 }
19
20 // Pop element from stack
21 char pop() {
22     if (top < 0) {
```

Test Input

stdin input (optional)

Output

Infix Expression: a+(b*c-
(d/e^f)*g)*h
Postfix Expression: abc*def^/g*-h*+

Your Fixed Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 char stack[MAX];
7 int top = -1;
8
9 // Function to push element into stack
10 void push(char ch) {
11     if (top < MAX - 1) {
12         stack[++top] = ch;
13     }
14 }
15
16 // Function to pop element from stack
17 void pop() {
18     if (top >= 0) {
19         top--;
20     }
21 }
22
```

Test Input

stdin input (optional)

Output

Balanced

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head = NULL, *temp, *n;
11
12     n = malloc(sizeof(struct Node));
13     n->data = 10; n->next = NULL; head = n;
14
15     n = malloc(sizeof(struct Node));
16     n->data = 20; n->next = NULL; head->next = n;
17
18     temp = head;
19     while (temp != NULL) {
20         printf("%d ", temp->data);
21         temp = temp->next;
22     }
23 }
```

Test Input

stdin input (optional)

Output

10 20

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX], top = -1;
5
6 void pop() {
7     if (top == -1)
8         printf("Underflow");
9     else
10        printf("Popped: %d", stack[top--]);
11 }
12
13 int main() {
14     stack[++top] = 10;
15     stack[++top] = 20;
16
17     pop();
18     return 0;
19 }
```

 Run Save Submit Fix

Test Input

stdin input (optional)

Output

Popped: 20

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *current, *prev = NULL, *next;
11     head = malloc(sizeof(struct Node));
12
13     head->data = 10;
14     head->next = NULL;
15
16     current = head;
17
18     while (current != NULL) {
19         next = current->next;
20         current->next = prev;
21         prev = current;
22         current = next;
23     }
```

Test Input

stdin input (optional)

Output

10

Your Fixed Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head = NULL, *new;
11
12     new = malloc(sizeof(struct Node));
13     new->data = 10;
14     new->next = head;
15     head = new;
16
17     printf("%d", head->data);
18     return 0;
19 }

```

Run

Save

Submit Fix

Test Input

stdin input (optional)

Output

10

Your Fixed Code

```

1 #include <stdio.h>
2 #define MAX 5
3
4 int Q[MAX], front = -1, rear = -1;
5
6 void dequeue() {
7     if (front == -1)
8         printf("Underflow");
9     else {
10         printf("Deleted: %d\n", Q[front++]);
11         if (front > rear)
12             front = rear = -1;
13     }
14 }
15
16 int main() {
17     int n, i;
18     printf("Enter number of elements: ");
19     scanf("%d", &n);
20
21     front = 0;
22     for (i = 0; i < n; i++)

```

Test Input

Enter number of elements: 3
10 20 30

Output

Enter number of elements: Deleted:
0

SET – 2

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4
5 #define MAX 100
6
7 // Stack structure and functions
8 int stack[MAX];
9 int top = -1;
10
11 void push(int val) {
12     if (top >= MAX - 1) return;
13     stack[++top] = val;
14 }
15
16 int pop() {
17     if (top < 0) return 0;
18     return stack[top--];
19 }
20
21 // Function to evaluate postfix expression
22 int evaluatePostfix(char* expression) {
```

Test Input

stdin input (optional)

Output

Result: 48

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int q[MAX], front = -1, rear = -1;
5
6 void enqueue(int x) {
7     if ((rear + 1) % MAX == front) return;
8     if (front == -1) front = 0;
9     rear = (rear + 1) % MAX;
10    q[rear] = x;
11 }
12
13 int dequeue() {
14     if (front == -1) return -1;
15
16     int x = q[front];
17
18     if (front == rear)
19         front = rear = -1;
20     else
21         front = (front + 1) % MAX;
22 }
```

Test Input

stdin input (optional)

Output

10

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int a[], int l, int r, int x) {
4     if (l > r) return -1;
5
6     int mid = l + (r - l) / 2;
7
8     if (a[mid] == x) return mid;
9     if (a[mid] > x)
10         return binarySearch(a, l, mid - 1, x);
11     return binarySearch(a, mid + 1, r, x);
12 }
13
14 int main() {
15     int a[] = {10, 20, 30, 40, 50}, x = 30;
16     int n = 5;
17
18     printf("Index: %d", binarySearch(a, 0, n - 1, x));
19     return 0;
20 }
```



Run



Save



Submit Fix

Test Input

stdin input (optional)

Output

Index: 2

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int deque[MAX], front = -1, rear = -1;
5
6 void insertFront(int data) {
7     if ((front == 0 && rear == MAX-1) || front == rear + 1)
8         printf("Deque Full");
9     else {
10         if (front == -1)
11             front = rear = 0;
12         else if (front == 0)
13             front = MAX - 1;
14         else
15             front--;
16
17         deque[front] = data;
18     }
19 }
20
21 int main() {
22     insertFront(10);
```

Test Input

stdin input (optional)

Output

Front: 10

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int queue[MAX], rear = -1;
5
6 void enqueue(int data) {
7     if (rear == MAX - 1)
8         printf("Queue Full");
9     else
10         queue[++rear] = data;
11 }
12
13 int main() {
14     enqueue(10);
15     enqueue(20);
16
17     printf("Queue: ");
18     for (int i = 0; i <= rear; i++)
19         printf("%d ", queue[i]);
20
21     return 0;
22 }
```

Test Input

stdin input (optional)

Output

Queue: 10 20

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *prev, *next;
7 };
8
9 void deleteNode(struct Node **head, struct Node *del) {
10     if (*head == NULL || del == NULL) return;
11
12     if (del->prev)
13         del->prev->next = del->next;
14     else
15         *head = del->next;
16
17     if (del->next)
18         del->next->prev = del->prev;
19
20     free(del);
21 }
22
23 int main() {
```

Test Input

stdin input (optional)

Output

10

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX], top = -1;
5
6 void pop() {
7     if (top == -1)
8         printf("Underflow");
9     else
10        printf("Popped: %d", stack[top--]);
11 }
12
13 int main() {
14     stack[++top] = 10;
15     stack[++top] = 20;
16     pop();
17     return 0;
18 }
```

 Run

 Save

 Submit Fix

Test Input

stdin input (optional)

Output

Popped: 20

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX], top = -1;
5
6 void push(int x) {
7     if (top == MAX - 1)
8         printf("Overflow");
9     else
10        stack[++top] = x;
11 }
12
13 int main() {
14     push(10);
15     printf("Top: %d", stack[top]);
16     return 0;
17 }
```

 Run

 Save

 Submit Fix

Test Input

stdin input (optional)

Output

Top: 10

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int q[MAX], front = -1, rear = -1;
5
6 void enqueue(int x) {
7     if ((rear + 1) % MAX == front)
8         printf("Queue Full");
9     else {
10         if (front == -1) front = 0;
11         rear = (rear + 1) % MAX;
12         q[rear] = x;
13         printf("Inserted: %d", x);
14     }
15 }
16
17 int main() {
18     enqueue(10);
19     return 0;
20 }
```



Run



Save



Submit Fix

Test Input

stdin input (optional)

Output

Inserted: 10

Your Fixed Code

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX], top = -1;
5
6 void peek() {
7     if (top == -1)
8         printf("Underflow");
9     else
10         printf("Top: %d", stack[top]);
11 }
12
13 int main() {
14     stack[++top] = 10;
15     stack[++top] = 20;
16
17     peek();
18     return 0;
19 }
```



Run



Save



Submit Fix

Test Input

stdin input (optional)

Output

Top: 20

